

Constraints in SQL Server with Examples:

In this article, I will discuss **Constraints in SQL Server** with examples. Please read our previous article where we discussed [SQL Server Data Types](#) in detail. As part of this article, we are going to discuss the following things in detail.

1. [What is Data Integrity?](#)
2. [What are SQL Server constraints?](#)
3. [Why do we need SQL Server constraints?](#)
4. [Different types of constraints in SQL Server?](#)
5. [Understanding NULL in SQL Server](#)
6. [What is Default, Not Null, Unique, and Check Constraint in SQL Server?](#)
7. [How to Impose Constraint on Table.](#)
8. [Imposing Constraint on Column.](#)
9. [What are composite constraints?](#)

Note: In the next article, I will discuss [Primary Key](#) and [Foreign Key](#) Constraints.

What is Data Integrity in SQL Server?

Data integrity means the data contained in the database is accurate, consistent, and reliable. To provide data integrity, RDBMS provides us with a set of integrity constraints that ensures that the data entered into the database is going to be accurate, consistent, and reliable. This is the reason why the end-user can trust the data.

What is a Constraint in SQL Server?

We can define the **SQL Server Constraint** as a property that can be assigned to a column or columns of a table. The SQL Server Constraints are mainly used to maintain data integrity.

Why do we need SQL Server Constraints?

The SQL Server Constraints are used to restrict the insertion of unwanted data in any columns i.e. they are mainly used to maintain data integrity. We can create the constraint on single or multiple columns of a table in SQL Server.

What are the different types of SQL Server Constraints available?

SQL Server supports six types of constraints for maintaining data integrity. They are as follows

1. **Default Constraint**
2. **UNIQUE KEY constraint**
3. **NOT NULL constraint**
4. **CHECK KEY constraint**
5. **PRIMARY KEY constraint**
6. **FOREIGN KEY constraint.**

Note: Constraints are imposed on columns of a table.

Before going to understand the constraints in SQL Server, first, we need to understand NULL in SQL Server.

Understanding NULL in SQL Server:

NULL represents the absence of data or value. It's neither ZERO nor EMPTY. In order to understand this, let us create the following Employee table. Here, EmployeeID is an int data type and it is not null. On the other hand, Name and Code fields are NULL types means they accept a NULL value.

Column Name	Data Type	Allow Nulls
EmployeeID	int	☐
Name	varchar(50)	☒
Code	int	☒

Once you create the above Employee table, let us add some data as shown below.

EmployeeID	Name	Code
1	AAAA	10001
2	BBBB	0
3		0

As you can see for employee id 2, the code is 0, and for employee id 3, the name is empty and the code is 0. Here, zero and empty do not represent NULL, You need to consider them as values.

Whenever you don't have any value, then you need to put it as Null. For example, let's say we have an employee whose code is not yet generated and supposed to come in later. Then, in that case, you can put a NULL in the code as shown below which means the code is absent at the moment.

EmployeeID	Name	Code
1	AAAA	10001
2	BBBB	0
3		0
4	DDDD	NULL

0 Considered as Value

Empty still considered as value

Data is Absence

Note: When you need to tell that the data is not present or absent, then you need to put NULL. The short-cut key to put NULL is (CTRL + 0).

Once we understand what exactly NULL is, now let us proceed to constraints.

Default Constraint in SQL Server

The Default constraint in SQL Server is used to fill the column with a default value that is defined during the creation of a table if the user does not supply any value while inserting the data. In simple words, we can say that Default constraints enable the SQL Server to insert a default value to a column when the user doesn't specify a value.

Note: IDENTITY and timestamp columns aren't associated with the default constraint.

Let us understand Default Constraint with an example. In order to understand this, first, create the Employee table by executing the following SQL Script.

```
CREATE TABLE Employee (  
    ID int NOT NULL,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255),  
    Age int,  
    City varchar(255) DEFAULT 'Mumbai',  
    DateOfBirth date DEFAULT GETDATE(),  
    Salary DECIMAL (18, 2) DEFAULT 5000.00  
)
```

As you can see in the above create table statement, here, we created the **City**, **DateOfBirth**, and **Salary** columns with the **Default Constraint**. So, while inserting the data into the Employee table, if the user does not specify the values for **City**, **DateOfBirth**, and **Salary** columns, then the default values we specified here are going to be stored in the store **City**, **DateOfBirth**, and **Salary** columns.

Let's insert a record into the Employee table by executing the following Insert statement.

Insert into Employee (ID, LastName, Age) values(1, 'Rout', 30)

As you can see, in the above insert statement, we have not specified the values for the **City**, **DateOfBirth**, and **Salary** columns, so they should be the default values. In order to make sure, it takes the default values, let's issue a select query as shown below.

Select * from Employee

Once you execute the above Select query. It gives us the below output and you can see the **City**, **DateOfBirth**, and **Salary** columns are filled with the default values.

ID	LastNa...	FirstNa...	Age	City	DateOfBirtg	Salary
1	Rout	NULL	30	Mumbai	2018-04-28	5000.00

NOT NULL Constraint in SQL Server:

When you want a column not to accept NULL then you need to apply the NOT NULL constraint to that column. That means this constraint is used to avoid NULL values but it accepted duplicate values into a column.

A table can contain any number of NOT NULL Constraints. We can apply the NOT NULL constraint on any data type column such as integer, character, money, etc.

Let us understand this with an example. To understand this concept, first, let us create the Customer table by executing the following SQL Script. As you can see, we applied the NOT NULL constraint to all three fields of the Customer table.

```
CREATE TABLE customer
(
    id      INT NOT NULL,
    name    VARCHAR(30) NOT NULL,
    mobno   CHAR(10) NOT NULL
)
```

Once you create the above Customer table, now try to execute the following SQL statements. The first, two statements are executed as expected as they don't have any null values.

```
INSERT customer VALUES (101, 'AA', '1111111111')--ALLOWED
INSERT customer VALUES (101, 'AA', '2222222222')--ALLOWED
INSERT customer VALUES (NULL, NULL, NULL)--NOTALLOWED
```

But when you try to execute the third insert statement, it will give you the below error.

Messages

Msg 515, Level 16, State 2, Line 18

Cannot insert the value NULL into column 'cid', table 'Test_DB.dbo.customer'; column does not allow nulls. INSERT fails.

The statement has been terminated.

Note: When we INSERT a null value into a column on which the NOT NULL constraint is imposed. The execution of the insert statement is terminated by displaying a user-friendly message telling the reason for termination and also specifying the database, the table, and the column where the problem got occurred.

UNIQUE Constraint in SQL Server:

When you want a column or columns not to accept any duplicate values, then you need to apply UNIQUE Constraint to that column or columns. That means the UNIQUE constraint is used to avoid duplicate values but it accepts a single NULL value in that column.

A table can contain any number of UNIQUE constraints. We can apply the UNIQUE constraint on any data type column such as integer, character, money, etc.

Syntax: **CREATE TABLE**<TABLE NAME>(<COLUMN NAME><DATA TYPE><CONSTRAINTKEY>.....)

Let us understand Unique Constraints with an example. In order to understand Unique Constraints, first, create the Customer table by executing the following SQL Script. As you can see, here, we applied the UNIQUE constraint to all three columns. It means none of the columns is going to accept the duplicate value.

```
CREATE TABLE Customer
(
    Id          INT UNIQUE,
    NAME        VARCHAR(30) UNIQUE,
    Emailid     VARCHAR(100) UNIQUE
)
```

Once you created the above table, now try to execute the below Insert statements.

```
INSERT customer VALUES (10, 'AA', 'pranayakumar777@gmail.com')
```

When we execute the above statement, it executes as expected. Now, try to execute the below SQL Statement.

```
INSERT customer VALUES (10, 'AA', 'pranayakumar777@gmail.com')
```

When we try to execute the above statement we will get the following error. This is because we are trying to insert duplicate values into the columns.

```
Msg 2627, Level 14, State 1, Line 14
Violation of UNIQUE KEY constraint 'UQ__customer__7EDA1EE623E989D8'. Cannot insert duplicate key in object 'dbo.customer'. The duplicate key value is (pranayakumar777@gmail.com).
The statement has been terminated.
```

Again try to execute the below SQL Statements.

```
INSERT customer VALUES (NULL, NULL, NULL)--ALLOWED
INSERT customer VALUES (11, 'BB', 'pranayakumar7@gmail.com')--ALLOWED
INSERT customer VALUES (NULL, NULL, NULL) -- NOT ALLOWED
```

When we execute the above query, the third insert statement will give us the below error

```
Messages
Msg 2627, Level 14, State 1, Line 28
Violation of UNIQUE KEY constraint 'UQ__customer__7EDA1EE623E989D8'. Cannot insert duplicate key in object 'dbo.customer'. The duplicate key value is (<NULL>).
The statement has been terminated.
```

If we insert a duplicate value into a column that contains a UNIQUE constraint, the statements get terminated by displaying an error message but that error message will not contain the column name where the violation got occurred, it only displays the constraint name, the table name, and the value that we want to insert into the unique column as shown in the above error message.

Note: UNIQUE constraint will accept only 1 NULL value.

UNIQUE KEY, PRIMARY KEY, CHECK, and FOREIGN KEY in SQL Server:

UNIQUE KEY, PRIMARY KEY, CHECK, and FOREIGN KEY Constraints are independent objects under the database on which they are created and linked with the column of the table. That means they have their own identity or name. This is the reason why if these four constraints are violated they will never tell the column where the violation occurred. They only display the constraint name, table name, and the value that we want to insert.

Actually, you should define the constraint name while creating the table. If you have not specified the constraint name while creating the table, then the SQL server will give a random name to that constraint which is not user-friendly. So it is always advisable to provide your own constraint name while creating the table for easy identification. The naming convention is something like below.

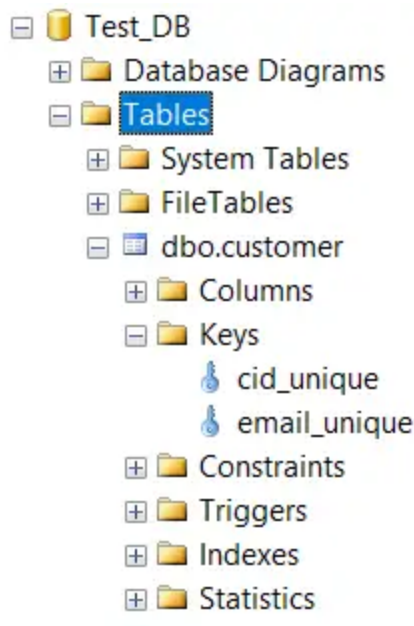
<columnname>_<constraint type>

Creating Constraint with own name:

Let us understand how to provide our own constraint name with an example. In order to understand this, please execute the below SQL Script which will create the Customer table. If you further notice, here we created two unique constraints by giving our own names such as cid_unique and email_unique.

```
CREATE TABLE customer
(
    Id          INT CONSTRAINT cid_unique UNIQUE,
    NAME        VARCHAR(30),
    Emailid     VARCHAR(100) CONSTRAINT email_unique UNIQUE
)
```

In the above create table statement we are explicitly providing the name for the constraints. Once you created the table you can find the constraint in the keys folder as shown below



Imposing Constraint in SQL Server:

We can impose constraints on a table in two different ways

1. **Imposing constraints on Column level**
2. **Imposing constraints on Table level**

In the first case, we need to provide the constraint information inside the column only whereas in the second case we first define all the columns, and then we need to define the constraints on the columns.

Note: We cannot impose a NOT NULL constraint at the table level. It is possible only for the rest of the four constraints.

Imposing Constraint on Column Level:

As you can see in the following Create Table statement, we have specified the **cid_unique** and **email_unique** unique constraints directly at the column level i.e. next to the column name.

```
CREATE TABLE customer
(
    Id          INT CONSTRAINT cid_unique UNIQUE,
    NAME        VARCHAR(30),
    Emailid     VARCHAR(100) CONSTRAINT email_unique UNIQUE
)
```

Imposing Constraint on Table Level:

As you can see in the following Create Table statement, first, we created all the columns and then we created the **cid_unique** and **email_unique** unique constraints and while creating the constraint we explicitly specified the column name to which the constraint belongs.

```
CREATE TABLE customer
(
    Id          INT,
    NAME        VARCHAR(30),
    Emailid     VARCHAR(100),
    CONSTRAINT cid_unique UNIQUE(Id),
    CONSTRAINT email_unique UNIQUE(Emailid)
)
```

Note: There is no difference in behavior whether the constraint is imposed at the table level or at the column level but if a constraint is imposed at the table level, then we have the advantage of imposing composite constraints.

What are Composite Constraints in SQL Server?

Whenever a constraint is created based on more than one column then it is called Composite Constraints in SQL Server. Let us understand the composite constraint with an example.

As you can see in the following Create table statement, we have imposed the **city_bc_unique** unique constraint at the table level and here we specified the City and Branchcode column. That means this constraint is a composite constraint as it is created based on two columns (City and Branchcode).

```
CREATE TABLE BranchDetails
(
    City          VARCHAR(50),
    Branchcode     VARCHAR(10),
    Branchlocation VARCHAR (30),
```

```
CONSTRAINT city_bc_unique UNIQUE(City, Branchcode)
)
```

Now insert some data to the BranchDetails table by executing the following insert statements.

```
Insert into BranchDetails (City, Branchcode, Branchlocation) values('Mumbai',
'xyz','abc') -- Allowed
Insert into BranchDetails (City, Branchcode, Branchlocation) values('Mumbai',
'abc','pqr') -- Allowed
Insert into BranchDetails (City, Branchcode, Branchlocation) values('Mumbai',
'xyz','pqr') --Not Allowed
```

When we execute the third statement, it will give us the following error. This is because the combination of Mumbai and xyz in the column City and Branchcode already exists and is considered duplicate values.

```
Msg 2627, Level 14, State 1, Line 16
Violation of UNIQUE KEY constraint 'city_bc_unique'. Cannot insert duplicate key in object 'dbo.BranchDetails'. The duplicate key value is (Mumbai, xyz).
The statement has been terminated.
```

Note: The drawback with a NOT NULL constraint is it will allow duplicate values whereas in the case of a UNIQUE constraint, it allows null values.

Check Constraint in SQL Server:

The Check Constraint is used to enforce domain integrity. Domain integrity ensures that the values going to store in a column must follow some defined rules such as range, type, and format.

In simple words, we can say that the Domain Integrity constraint enforces the valid entries for a given column value by restricting the type of the value, the format of the data, or the range of possible values. Let's understand this with an example.

```
CREATE TABLE Employee
(
    Emp_id          INT NOT NULL CHECK(Emp_id BETWEEN 0 AND 1000),
    Emp_name        VARCHAR(30) NOT NULL,
    Entered_date    DATETIME NOT NULL CHECK(Entered_date <= CURRENT_TIMESTAMP),
    Dept_no         INT CHECK(Dept_no > 0 AND Dept_no < 100)
)
```

We need to use the Check Constraints in the SQL server to limit the range of possible values of a column.

The check constraints can be created at two different levels

1. **Column-Level Check Constraints:** When we create the check constraints at the column level then they are applied only to that column of the table.
2. **Table-level Check Constraints:** When we create the check constraints at the table level, then it can be referred from any column within that table.

A table can contain any number of check constraints and will apply to any column data type like integer, character, decimal, date, etc.

Example: Let's insert some data to understand this concept

```
INSERT INTO Employee
    (Emp_id,
     Emp_name,
     Entered_date,
     Dept_no)
VALUES (1,
       'Pranaya',
       '2018-04-28 12:18:46.813',
       10) -- Allowed

INSERT INTO Employee
    (Emp_id,
     Emp_name,
     Entered_date,
     Dept_no)
VALUES (1001,
       'Pranaya',
       '2018-04-28 12:18:46.813',
       20) --Not Allowed
```

When we execute the above query it will give us the below error

Msg 547, Level 16, State 0, Line 10
The INSERT statement conflicted with the CHECK constraint "CK_Employee_Emp_id_32E0915F". The conflict occurred in database "Test_DB", table "dbo.Employee", column 'Emp_id'.
The statement has been terminated.

```
INSERT INTO Employee
    (Emp_id,
     Emp_name,
     Entered_date,
     Dept_no)
VALUES (2,
       'Pranaya',
       '2018-04-30 12:18:46.813',
       10) - Not Allowed
```

It will give us the below error when we execute the above statement

Msg 547, Level 16, State 0, Line 19
The INSERT statement conflicted with the CHECK constraint "CK_Employee_Entere__33D4B598". The conflict occurred in database "Test_DB", table "dbo.Employee", column 'Entered_date'.
The statement has been terminated.

```
INSERT INTO Employee
    (Emp_id,
     Emp_name,
     Entered_date,
     Dept_no)
VALUES (3,
       'Pranaya',
       '2018-04-27 12:18:46.813',
```

101) --Not Allowed

When we execute the above query it will give us the below error

Msg 547, Level 16, State 0, Line 28
The INSERT statement conflicted with the CHECK constraint "CK_Employee_Dept_n_34C8D9D1". The conflict occurred in database "Test_DB", table "dbo.Employee", column 'Dept_no'.
The statement has been terminated.

In the next article, I will discuss the [Primary Key Constraints in SQL Server](#) with examples. Here, in this article, I try to explain **Constraints in SQL Server** with examples. We discussed SQL server Constraints such as **Default, Not Null, Unique, and Check Constraints** in this article.